CRM9250NDev API Description

Content

1 Instructions for Use 3
2 API status code 4
3 Data structure 5
eCRM9250N_SubType 5
3.1 tDevReturn 5
3.2 tCRM9250N_InitSettingEx 5
3.3 tCRM9250N_InOutUnitsInfoEx 6
3.4 tCRM9250N_InitInfoEx 6
3.5 tCRM9250N_CassetteInfoEx 6
3.6 tCRM9250N_BinInfo 7
3.7 tCRM9250N_CassetteStatusEx 7
3.8 tCRM9250N_BinStatus 7
3.9 tCRM9250N_DevStatusEx 8
3.10 tCRM9250N_DenomAndNum 8
3.11 tCRM9250N_DenomNumInfo 9
3.12 tCRM9250N_CashCount 9
3.13 tCRM9250N_StoreMoneyEx 10
3.14 tCRM9250N_CancelDepositEx 10
3.15 tCRM9250N_DispenseInfoEx 10
3.16 tCRM9250N_RetractInfoEx 11
eCRM9250N_RetractPos 11
3.17 tCRM9250N_DenominationInfo 12
3.18 tGRGNoteGoDest 12
3.19 tGRGNoteDetails 12
3.20 tGRGSegSnData 13
3.21 tGRGSegOneSnInfo 13
3.22 tGRGNoteOCRInfo 13
3.23 tCRM9250N_PDLProgress 14
3.24 tCRM9250N_DevVersionInfoEx 14
3.25 tCRM9250N_DenominationTable 15
3.26 tCRM9250N_LastInOutUnitsInfo 15
3.27 tCRM9250N_DenomDetails 15
3.28 tCRM9250N_ReqDispInfo 15
4 API 16
4.1 hOpenLogicDevice 16
4.2 vCloseLogicDevice 16
4.3 iSetCommPara 16
4.4 iCloseComm 16
4.5 iInitSettingEx 17
4.6 iInitEx 17
4.7 iGetCassetteInfoEx 17

4.8 iSetCassetteInfoEx 17
4.9 iPowerOnInitEx 18
4.10 iGetDevStatusEx 18
4.11 iGetLastDevStatusEx 18
4.12 iPrepareNextTransaction 19
4.13 iPrepareDeposit 19
4.14 iOpenShutter 19
4.15 iCloseShutter 19
4.16 iGetLastTranDetail 20
4.17 iCashCount 20
4.18 iStoreMoneyEx 20
4.19 iCancelDepositEx 21
4.20 iDispenseEx 21
4.21 iRetractEx 21
4.22 iGetDenominationTable 21
4.23 iGetNvConnected 22
4.24 iGetNvNotesInfoEx 22
4.25 iFirmwareDownload 23
4.26 iGetCurFileProgress 23
4.27 iGetDevVersionEx 23
4.28 iGetCassetteDenomDetailsEx 23
5 Recommended Flow 24
5.1 Power on 24
5.2 Dispense 25
5.3 Deposit 26

1 Instructions for Use

1.1 driver+dunite

When developer use GRG's SDK or toolplus, developer need to install the Dunite platform and GRGUSBDriver first.

1.2 All dll and .ini file should be used together in the same directory.CRM9250NDev.dll is the main dll.

1.3 GRGDTATM_CommCfg.ini is used to configure the communication mode.

CommType =3 // 1-COM; 3-USB.

ComID =2 //Serial port number, This value is invalid for USB communication.

1.4 The general format of API.

int iCommandName(HANDLE p_hDevHandle, (……) , tDevReturn *p_psStatus);

Returned value of API. SUCCESS: 0 or FAIL:1 (or non-zero).

Note: p_psStatus may return a warning code even if SUCCESS:0 is returned.

Code example:

tDevReturn l_sStatus[8] = {0};

int ret= iCommandName((……) , l_sStatus);

Note: tDevReturn must be array size 8，tDevReturn.iLogicCode (Reference API status code).

1.5 When the APP calls DLL, Log files will be generated. When you have SDK problems, send Log (DevDriverLog file + CRM9250NLog file) to GRG for analysis and specify the time point.

2 API status code

Status codes are divided into physical codes and logical codes. Physical code refers to the real return code of the device, which describes a specific real state of the device.Logical code refers to the return code after analysis and classification of physical codes, reflecting a certain type of device status with the same nature.The relationship between them is that a logical code may correspond to multiple physical codes, or it may be a one-to-one correspondence.
The iLogicCode of tDevReturn is required for software development.
Refer to doc 《CRM9250N LogicalCode.xls》

3 Data structure

eCRM9250N_SubType

```
typedef enum
{
    CRM9250N_SUBTYPE_UNKNOWN = 0,//Unknown model
    CRM9250N_SUBTYPE_A = 'A',    //CRM9250N Model-A
    CRM9250N_SUBTYPE_B = 'B',    //Model-B
    CRM9250N_SUBTYPE_C = 'C',    //Model-C(4 RC slot+1 AC slot+LRB)
    CRM9250N_SUBTYPE_D = 'D',    //Model-D(2 DRC slot+1 AC slot+LRB)
    CRM9250N_SUBTYPE_E = 'E'     //Model-E
} eCRM9250N_SubType;
```

3.1 tDevReturn

```
typedef struct
{
    int         iLogicCode;   //Logical code
    int         iPhyCode;     //Physical code
    int         iHandle;      //Handling methods: 0-No 1-Initializing 2 -resend
    int         iType;        //Status code type: 0-Warning 1-Error
    char acDevReturn[128]; //Reserve
    char acReserve[128];     //Reserve
} tDevReturn;
```

3.2 tCRM9250N_InitSettingEx

```
typedef struct tCRM9250N_InitSettingEx
{
    BYTE    abyCassetteDisable[16];//[in]Disable each casette(RC1-RC16) 0- Enable 1- Disable
    WORD    awAcceptableDenom[256]; //[in]Acceptable denomination ID (empty: all not
accepted)
    char cSubType;                  //[out]Machine model-eCRM9250N_SubType
    BYTE    byAtcMode;              //[in]ATC mode: 0-only audit; 1-audit and RC
    BYTE    abyDRCCassetteCfg[16];      //[in]DRC slot configuration
    BYTE    byAcMode; //[in]AC mode:0-used as AC; 1-used as RC(Only supported by
CRM9250N Model-D)
```

```
    BYTE    byWorkingMode;          // [in] Set 0-CRM Machine Mode
    BYTE    byDcAccepetAllDenom;  // [in] Set 1-Receive all denominations in the
"awAcceptableDenom"
    BYTE    byInitCfg; // [in] Set 0-Normal mode. During initialization, check the AC box.
    BYTE    abyReserved[17];   //Reserved
} tCRM9250N_InitSettingEx;
```

3.3 tCRM9250N_InOutUnitsInfoEx

A positive number means in and a negative number means out

```
typedef struct tCRM9250N_InOutUnitsInfoEx
{
    int   aiRcInOutNum[16];      // [out]The number of notes in/out of RC(RC1-RC16)
    int   iAcInOutNum[2];   // [out]The number of notes in/out of AC
    int   iLrbInOutNum;             // [out]The number of notes in/out of LRB
    int   iAtcInOutNum;       // [out]The number of notes in/out of ATC
    int   iNeInOutNum;       // [out]The number of notes in/out of NE
    int   iNfInOutNum;// [out]The number of notes in/out of NF
    BYTE      abyReserved[32];              //Reserved
} tCRM9250N_InOutUnitsInfoEx;
```

3.4 tCRM9250N_InitInfoEx

```
typedef struct tCRM9250N_InitInfoEx
{
    tCRM9250N_InOutUnitsInfoEx    sInOutUnitsInfo;//[out] The number of notes in/out of
each component(inaccuracy)
    BYTE    byMode;          //set 0-normal speed
    BYTE    byNEMode;    // set 0-During initialization,NE removes the notes.
    BYTE    abyReserved[30];//Reserved
} tCRM9250N_InitInfoEx;
```

3.5 tCRM9250N_CassetteInfoEx

```
typedef struct    tCRM9250N_CassetteInfoEx
{
    BYTE      abyCassetteDisable[16];//[in]Disable each casette(RC1-RC16) 0- Enable 1-
Disable
    tCRM9250N_BinInfo      sAcCassette[2];    //AC cassette information
    tCRM9250N_BinInfo      sAtcCassette;     //ATC cassette information
```

```
    tCRM9250N_BinInfo      asRcCassette[16];//RC&DC cassette information
    BYTE                   abyReserved[32]; //Reserved
} tCRM9250N_CassetteInfoEx;
```

3.6 tCRM9250N_BinInfo

```
typedef struct tCRM9250N_BinInfo
{
    BYTE    abyLogID[8];     //Cassette logic ID
    BYTE    byCountryCode;   //Reserved
    BYTE    byCassetteType;
//Cassette type:1-RC&ATC,2-AC,3-DC,4-NRC,5-NDC,6-DRC,9-DAC
    WORD    wNoteType;//Cassette denomination. For RC is the denomination code;
Meaningless for AC; Must be 0xFFF0 for DC
    BYTE    byFunction;  //Cassette function for RC: 0x00-loop function,0xF0 only dispense
0xFF-only store; must be 0xF3 for ATC; must be 0xF2 for AC; must be 0xF1 for DC
    BYTE    byStoreType;//0-RC deposits according to denomination
    WORD    wRCAsDC;// 0x8000-RC is used as DC(byFunction must be F1); 0-by default
    BYTE    abyReserved[30];//Reserved
} tCRM9250N_BinInfo;
```

3.7 tCRM9250N_CassetteStatusEx

```
typedef struct tCRM9250N_CassetteStatusEx
{
    tCRM9250N_BinStatus          asRcStatus[16];    //[out] RC status.RC1-RC16
    tCRM9250N_BinStatus          sAtcStatus;        //[out] ATC status
    tCRM9250N_BinStatus          sAcStatus[2];  //[out] AC status
    tCRM9250N_BinStatus          sLrbStatus;
//[out] LRB status,byExist(effective),byStatus(reserved)
} tCRM9250N_CassetteStatusEx;
```

3.8 tCRM9250N_BinStatus

```
typedef struct   tCRM9250N_BinStatus
{
    BYTE    byStatus;//[out]Cassette status:0- Normal 1- Full 2- will be full 3- will be empty 4-
empty 255- Abnormal
    BYTE    byExist;//[out]Cassette exists:0-does not exist 1-exists
} tCRM9250N_BinStatus;
```

3.9 tCRM9250N_DevStatusEx

```
typedef struct tCRM9250N_DevStatusEx
{
    BYTE    byShutterAreaHasNotes;// [out]Whether the shutter area has note. 0-No 1-Yes
    BYTE    byNfHasNotes;      // [out]Whether the NF has note. 0-No 1-Yes
    BYTE    byNfNotesWrongPosition;    // [out]Whether the notes of NF are placed correctly.0-position correct;1-position incorrect
    BYTE    byNeHasNotes;      // [out] Whether NE has notes: 0-No, 1-Yes
    BYTE    byShutterOpen;     // [out]Whether the shutter is open. 0-closed 1-open
    BYTE    byNvOpen;                  // [out]NV Status.0-closed 1-open
    BYTE    byLowerChannelOpen;        // [out]Lower Channe status.0-closed 1-open
    BYTE    byUpperUnitDragOut;        // [out]Upper unit.0-Push in; 1-Drag out
    BYTE    byUpperUnitInPos;          // [out]Upper unit.0-Not in place 1-in place
    BYTE    byLowerUnitDragOut;        // [out]Lower unit.0-Push in; 1-Drag out
    BYTE    byLowerUnitInPos;          // [out]Lower unit.0-Not in place 1-in place
    BYTE    byCassettePlugRemove;      // [out]The cassette is inserted/pulled out.0-No 1-Yes;If a cassette is inserted/pulled out,need to call "iGetCassetteInfoEx", and then this bit becomes 0
    tCRM9250N_CassetteStatusEx    sCassetteStatus;          // [out]Cassette status
    BYTE    abyDevStatusInfo[30];      // [out]Device status information 7 bytes retained.
    BYTE    abySensorStatusInfo[45];   // [out]Sensor status
    BYTE    byRC1PlugRemove;           //The RC1 is inserted/pulled out.0-No 1-Yes
    BYTE    byRC2PlugRemove;           //The RC2 is inserted/pulled out.0-No 1-Yes
    BYTE    byRC3PlugRemove;           //The RC3 is inserted/pulled out.0-No 1-Yes
    BYTE    byRC4PlugRemove;           //The RC4 is inserted/pulled out.0-No 1-Yes
    BYTE    byACPlugRemove;            //The AC is inserted/pulled out.0-No 1-Yes
    BYTE    byATCPlugRemove;           //The ATC is inserted/pulled out.0-No 1-Yes
    BYTE    bySpeedStrategy;           //0-normal
    BYTE    byRC9PlugRemove;           //The RC9 is inserted/pulled out.0-No 1-Yes
    BYTE    byRC10PlugRemove;          //The RC10 is inserted/pulled out.0-No 1-Yes
    BYTE    byRC11PlugRemove;          //The RC11 is inserted/pulled out.0-No 1-Yes
    BYTE    byRC12PlugRemove;          //The RC12 is inserted/pulled out.0-No 1-Yes
    BYTE    byAC2PlugRemove;           //The AC2 is inserted/pulled out.0-No 1-Yes
    BYTE    byOutPosHasNotes;          //The NF-Outlet export has notes
    BYTE    abyReserved[19];           //Reserved
} tCRM9250N_DevStatusEx;
```

3.10 tCRM9250N_DenomAndNum

```
typedef struct tCRM9250N_DenomAndNum
```

```
{
    WORD    wCode;                          // [out]Note denomination ID
    int     iNum;                           // [out]The number of note
} tCRM9250N_DenomAndNum;
```

3.11 tCRM9250N_DenomNumInfo

```
typedef struct tCRM9250N_DenomNumInfo
{
    tCRM9250N_DenomAndNum       asDenomNumL4A[256];// [out]ECB-4A note denomination and quantity information, otherwise retained，Otherwise, it represents all the note denomination and quantity information
    tCRM9250N_DenomAndNum       asDenomNumL4B[256];// [out]ECB-4B note denomination and quantity information, otherwise retained
    tCRM9250N_DenomAndNum       asDenomNumL3[256];// [out]ECB-L3 note denomination and quantity information, otherwise retained
    tCRM9250N_DenomAndNum       asDenomNumL2[256];// [out]ECB-L2 note denomination and quantity information, otherwise retained
    tCRM9250N_DenomAndNum       asDenomNumReserved1[256];// [out]Reserved
    tCRM9250N_DenomAndNum       asDenomNumReserved2[256];// [out]Reserved
    int    iUnknownCount;    // [out]Number of unidentified notes
} tCRM9250N_DenomNumInfo;
```

3.12 tCRM9250N_CashCount

```
typedef struct tCRM9250N_CashCount
{
    int    iDepositLimit;//[in]Deposit limit, if value<=0, ini file shall prevail
    WORD    awUnacceptableDenom[256];//[in]Unacceptable denomination ID(empty:all accepted),only this count is valid
    int    iTotalCount;//[out]This counting operation, the number of notes entering NE, accurate
    int    iRejectCount;  //[out]The number of rejected notes in this counting operation, inaccuracy
    tCRM9250N_DenomNumInfo      sDenomNumInfo;//[out]Information about the notes entering NE
    DWORD       dwAmountLimit;   //[in]Deposit amount limit
    BYTE        abyReserved[28];   //Reserved
} tCRM9250N_CashCount;
```

3.13 tCRM9250N_StoreMoneyEx

```
typedef struct tCRM9250N_StoreMoneyEx
{
    BYTE    abyCassetteDisable[16]; //[in]Disable each casette(RC1-RC16) 0- Enable 1-Disable
    BYTE    abyCassettePriority[16]; //[in]RC priority (RC1-RC16), 0x01-0x0F, where lower values have higher priority
    WORD    awUnacceptableDenom[256];//[in]Unacceptable denomination ID(empty:all accepted),only this StoreMoney is valid
    tCRM9250N_InOutUnitsInfoEx    sInOutUnitsInfo;//[out]The number of notes in/out of each component(inaccuracy)
    tCRM9250N_DenomNumInfo        sDenomNumInfo;  //[out]Information of notes deposited into the cassettes
    BYTE    byRejectPos;  //[in]Deposit rejection configuration 1: AC1,2:AC2,3:LRB
    BYTE    byNvMode;                   // [in]0-Verification (default)   1-No verification
    BYTE    abyReserved[30];//Reserved
} tCRM9250N_StoreMoneyEx;
```

3.14 tCRM9250N_CancelDepositEx

```
typedef struct tCRM9250N_CancelDepositEx
{
    tCRM9250N_InOutUnitsInfoEx    sInOutUnitsInfo;    // [out]The number of notes in/out of each component(inaccuracy)
    tCRM9250N_DenomNumInfo    sDenomNumInfo;    // [out]Information of notes entering the cassettes(only meaningful if the device is performing ECB)
    BYTE    abyReserved[32];        // Reserved
} tCRM9250N_CancelDepositEx;
```

3.15 tCRM9250N_DispenseInfoEx

```
typedef struct tCRM9250N_DispenseInfoEx
{
    BYTE    byDispType;         // [in] set = 1 -Dispense by RC
    tCRM9250N_ReqDispInfo    asReqDispInfo[16];  // [in]Request dispense information
    BYTE    abyCassetteDisable[16];  // [in]Disable each casette(RC1-RC16) 0- Enable 1-Disable
    int    iTotalCount;    // [out]Number of notes dispense to NF, accurate
    tCRM9250N_DenomAndNum    asDenomNum[256];    // [out]Information of notes dispense to NF
```

```
    int    aiDispensedInfo[16]; // [out]The number of notes dispense from each RC to NF may be inaccurate (RC1-RC16)
    int    aiRejectInfo[16];    // [out]The number of rejected notes in each RC, may be inaccurate (RC1-RC16)
    tCRM9250N_InOutUnitsInfoEx    sInOutUnitsInfo;    // [out]The number of notes in/out of each component, iNfInOutNum accurate, aiRcInOutNum may not be accurate
    BYTE    byNFNotesPolicy;  // Reserved，set = 0
    BYTE    byRejectToAc;      // [in]1-AC1,2-AC2,3-LRB Select the destination of dispense rejection
    BYTE    byCountEnable;// [in]0-Dispense function,1-cleaning machine function
    BYTE    byCountError; // [in]Error if the number of notes is insufficient when Dispense.(status code)0-Error;1-Warning;
    BYTE    byCountNVPolicy; // [in]0-Enable NV identification; 1-Disable NV identification
    BYTE    abyReserved[28];   // Reserved
} tCRM9250N_DispenseInfoEx;
```

3.16 tCRM9250N_RetractInfoEx

```
typedef struct tCRM9250N_RetractInfoEx
{
    BYTE    byRetractType;      // [in]1-Retract NF.2-Retract NE
    BYTE    byRetractPos;  // [in]The destination of the retract,see eCRM9250N_RetractPos
    BYTE    abyCassetteDisable[16]; // [in]Disable each RC (RC1-RC16) 0- Enable 1- Disable
    BYTE    abyCassettePriority[16]; // [in]RC priority (RC1-RC16),value 0x01-0x0F the lower the number, the higher the priority
    int         iTotalCount;    // [out]The number of retract notes, which may be inaccurate
    tCRM9250N_DenomNumInfo      sDenomNumInfo;   // [out]The denomination and quantity information of the retract note
    tCRM9250N_InOutUnitsInfoEx    sInOutUnitsInfo;    // [out]The number of notes in/out of each component,may be inaccurate
    BYTE    abyRetractFlag; // [in] set 0
    BYTE    abyReserved[31];   //Reserved
} tCRM9250N_RetractInfoEx;
```

eCRM9250N_RetractPos

```
typedef enum
{
    RETRACT_POS_LRB  = 1,  //All notes are retracted to LRB
    RETRACT_POS_AC   = 2,  //All notes are retracted to AC
    RETRACT_POS_NE   = 3,  //All notes are retracted to NE
    RETRACT_POS_AC2  = 4,  //All notes are retracted to AC2
```

```
    RETRACT_POS_RC     = 5,  //All notes are retracted to RC,but unqualified notes go into AC
    RETRACT_POS_ACEX = 6,//All notes are retracted to extension AC
    RETRACT_POS_DC     = 7,  //All notes are retracted to DC
    RETRACT_POS_RCLRB = 8      //All notes are retracted to RC,but unqualified notes go
into LRB
} eCRM9250Ns_RetractPos;
```

3.17 tCRM9250N_DenominationInfo

```
typedef struct tCRM9250N_DenominationInfo
{
    WORD        wDenomCode;      // [out] Denomination ID
    WORD        wLength;       // [out] Length
    WORD        wWidth;        // [out] Width
    WORD        wThickness;    // [out] Thickness
} tCRM9250N_DenominationInfo;
```

3.18 tGRGNoteGoDest

```
typedef struct tGRGNoteGoDest
{
    BYTE    iType;          //Destination of notes (1-cassette; 2-NF; 3-NE; 4-URJB)
    char    acDest[8];      //If iType==1, it is the cassett ID
}tGRGNoteGoDest;
```

3.19 tGRGNoteDetails

```
typedef struct tGRGNoteDetails
{
    UINT                uiID;                   //Reserved
    SYSTEMTIME          sSysTime;               //Reserved
    char            acJournalNo[32];   //Reserved
    char            acAccountNo[48];  //Reserved
    BYTE                byTimes;            //Reserved
    WORD                wNoteIndex;             //Note index
    BYTE                byValidation;       //0-real note;other-Not real note
    BYTE                byFaceWay;              //note face
    WORD                wDenomination;          //Denomination ID
    tGRGNoteGoDest  sNoteGoDest;        //Destination of notes
    char            acImageFileName[64];//Note serial number image filename
```

```
    char        acSerialNo[32];     //Serial number of the note
    char        acMac[8];           //Image Mac code
    BYTE        byMatchCount;       //Reserved
}tGRGNoteDetails;
```

3.20 tGRGSegSnData

```
typedef struct tGRGSegSnData
{
    DWORD adwData[32];      //Image crown word number dot matrix data
}tGRGSegSnData;
```

3.21 tGRGSegOneSnInfo

```
typedef struct tGRGSegOneSnInfo
{
    WORD        wNum;           //Number of characters, same as wCharNum
    WORD        wHeight;        //Height of each image character, maximum 32
    WORD        wWidth;         //Width of each image character, maximum 32
    tGRGSegSnData   asSnData[12]; // Dot matrix data of up to 12 characters.
}tGRGSegOneSnInfo;
```

3.22 tGRGNoteOCRInfo

```
typedef struct tGRGNoteOCRInfo
{
    WORD                wNoteIndex;             //Note index
    tGRGSegOneSnInfo    asSnInfo;           //Dot matrix data
    BYTE                abyInfoErrCode[6];//Note identification error code
    BYTE                abyRejectErrCode[6];//Rejection error code
    BYTE                byQuality;              //Reserved
    char                acImageNameFront[260];  //Full image filename _ Front
    char                acImageNameBack[260];   //Full image filename _ reverse
    BYTE                abyReserved[32];
}tGRGNoteOCRInfo;
```

3.23 tCRM9250N_PDLProgress

```
typedef struct tCRM9250N_PDLProgress
{
    char      acFileName[260];   //The current download filename
    int           iTotalSize;      //File size
    int           iCurPos;  //The currently downloaded location
    BYTE          abyReserved[32]; //Reserved
} tCRM9250N_PDLProgress;
```

3.24 tCRM9250N_DevVersionInfoEx

```
typedef struct tCRM9250N_DevVersionInfoEx
{
    char      cSubType;      // [out]Type,see eCRM9250N_SubType
    char      acFwMainBoard[32];     // [out]Main board Firmware version
    char      acFwUpperBoard[32];    // [out]Upper board Firmware version
    char      acFwAc[2][32];      // [out]AC1/AC2 Firmware version
    char      acFwLrb[8];    // [out]Reserved
    char      acFwRc[16][32];    // [out] RC1-16 Firmware version
    char      acFwAtc[32];        // [out] ATC Firmware version
    char      cHwNvInfo[32];     // [out] NV Hardware Information
    char      acFwDSP[32];        // [out] DSPFirmware version
    char      acFwGraphicsFPGA[32];// [out] Image FPGA version
    char      acFwArithmetic[32];      // [out] Algorithm version
    char      acFwMagneticArithmetic[32];// [out] Magnetic algorithm version
    char      acFwMagneticFPGA[32];      // [out] Magnetic FPGA version
    char      acFwFullFormatThickness[32];     // [out] Thickness version
    char      acFwUltravioletArithmetic[32];     // [out] Ultraviolet Arithmetic version
    char      acFwDSPGraphicsArithmetic[32];// [out] DSP image algorithm version
    char      acFwFSNArithmetic[32];// [out] Crown font size algorithm version
    char      acFwECBSoftware[32]; // [out] ECB software version
    char      acHwInfo[32];// [out] Hardware version information
    char      acHwNv[32];  // [out] NV hardware version
    char      acHwUnits[32];// [out] Device hardware version
    char      bySecFlag;// [out] Security mark, 0-unsupported, 2-ecurity 2.0
    char      acFwDriverBoard[32];   // [out] Driver board version
    char      byFsnCertification;        // [out] Reserved
    char     byCassetteLogicNums;       // [out] Reserved
    char      acReserved[253];    // Reserved
} tCRM9250N_DevVersionInfoEx;
```

3.25 tCRM9250N_DenominationTable

```
typedef struct    tCRM9250N_DenominationTable
{
    tCRM9250N_DenominationInfo    asDenominationInfo[512];    // [out]Device support currency information
    BYTE            abyReserved[32];                // Reserved
} tCRM9250N_DenominationTable;
```

3.26 tCRM9250N_LastInOutUnitsInfo

```
typedef struct tCRM9250N_LastInOutUnitsInfo
{
    int    aiRcInOutNum[16];// [out]Number of incoming and outgoing notes of RC (RC1~RC16)
    int    iAcInOutNum;       // [out]Number of incoming and outgoing notes of AC
    int    iLrbInOutNum;      // [out]Number of incoming and outgoing notes of LRB
    int    iAtcInOutNum;      // [out]Number of incoming and outgoing notes of ATC
    int    iNeInOutNum;       // [out]Number of incoming and outgoing notes of NE
    int    iNfOutNum;         // [out]Number of outgoing notes of NF
    int    iNfInNum;          // [out]Number of incoming notes of NF
    BYTE    byPowerOff;   // [out]Whether the power is off marks 0-No.0xFF-Power off
    BYTE    abyReserved[31];// Reserved
} tCRM9250N_LastInOutUnitsInfo;
```

3.27 tCRM9250N_DenomDetails

```
typedef struct tCRM9250N_DenomDetails
{
    WORD    wDenomCode;       // Denomination code. 0:unavailable denomination
    int         iNoteCount;    // count value
    BYTE    byNoteType;   //Note type: 2-Genuine(FIT) 3-L2 4-L3 5-L4B
} tCRM9250N_DenomDetails;
```

3.28 tCRM9250N_ReqDispInfo

```
typedef struct tCRM9250N_ReqDispInfo
{
    WORD    wDenomOrCassetteInfo;       // RC number
    int    iNum;     // The number of notes required
} tCRM9250N_ReqDispInfo;
```

4 API

4.1 hOpenLogicDevice

This API returns a different handle when passed in a different logical name.The software needs to save this handle and pass it in when calling other APIs. So this is the first API you should call.

HANDLE hOpenLogicDevice (const char* p_pcLogicDevName);
p_pcLogicDevName[in]: Pass the device logical name. It is the same as the node name[CRM9250NDev] of GRGDTATM_CommCfg.ini
Code Example:
HANDLE hret = hOpenLogicDevice("CRM9250NDev");

4.2 vCloseLogicDevice

Close the handle. This is the final step

void vCloseLogicDevice(HANDLE p_hDevHandle);
p_hDevHandle[in] Device handle.

4.3 iSetCommPara

Based on the logical device name set by the user, read the parameters to be set for communication from GRGDTATM_CommCfg.ini, set the parameters for communication of the device, and establish a connection for communication with the device

int iSetCommPara(HANDLE p_hDevHandle, tDevReturn* p_psStatus);
p_hDevHandle[in] Device handle.
p_psStatus [out] Return status structure.

4.4 iCloseComm

Turn off communication

int iCloseComm(HANDLE p_hDevHandle, tDevReturn* p_psStatus)
p_hDevHandle[in] Device handle.
p_psStatus [out] Return status structure.

4.5 iInitSettingEx

Initial settings.

int iInitSettingEx(HANDLE p_hDevHandle, tCRM9250N_InitSettingEx *p_psInitSetting, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psInitSetting[in out] Initializes the setup
p_psStatus [out] Return status structure.

4.6 iInitEx

Initializing the machine. After the device fails, this API needs to be called to try to reset, and the notes in the channel are sent to the NF. However, if part of the note has entered the cassette, it goes directly into the cassette.

int iInitEx(HANDLE p_hDevHandle, tCRM9250N_InitInfoEx *p_psInitInfo, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psInitInfo[out] Initialization information
p_psStatus [out] Return status structure.

4.7 iGetCassetteInfoEx

Get cassette information.

int iGetCassetteInfoEx(HANDLE p_hDevHandle,tCRM9250N_CassetteInfoEx *p_psCassetteInfo,tCRM9250N_CassetteStatusEx *p_psCassetteStatus,tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psCassetteInfo[in out] Cassette information.
p_psCassetteStatus[out] Cassette status
p_psStatus [out] Return status structure.

4.8 iSetCassetteInfoEx

Set cassette information.

int iSetCassetteInfoEx(HANDLE p_hDevHandle, const tCRM9250N_CassetteInfoEx *p_psCassetteInfo, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle

p_psCassetteInfo[in out] Cassette information.
p_psStatus [out] Return status structure.

4.9 iPowerOnInitEx

Combining commands = iInitSettingEx + iGetCassetteInfoEx + iInitEx
It is recommended that every time the device is powered on/after the device has physical operation (The cassette is inserted/pulled out, etc.).Call the composite API.

int iPowerOnInitEx(HANDLE p_hDevHandle,tCRM9250N_InitSettingEx *p_psInitSetting, tCRM9250N_CassetteInfoEx *p_psCassetteInfo,tCRM9250N_CassetteStatusEx *p_psCassetteStatus, tCRM9250N_InitInfoEx *p_psInitInfo,tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psInitSetting[in out] Initializes the setup
p_psCassetteInfo[in out] Cassette information.
p_psCassetteStatus[out] Cassette status
p_psInitInfo[out] Initialization information
p_psStatus [out] Return status structure.

4.10 iGetDevStatusEx

Getting machine status, if tCRM9250N_DevStatusEx. ByCassettePlugRemove value is 1, shows the cassette been pull out the insert, you need to call “iGetCassetteInfoEx”, after reading the cassette information, byCassettePlugRemove value becomes 0.

int iGetDevStatusEx(HANDLE p_hDevHandle, tCRM9250N_DevStatusEx *p_psDevStatus, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psDevStatus[out] Machine status
p_psStatus [out] Return status structure.
Note:
The difference between **iGetDevStatusEx** and **iGetLastDevStatusEx** is that the former queries the status by communicating with the machine, while the latter reads the current status in memory (which is automatically updated internally by the dll after any action is performed).

4.11 iGetLastDevStatusEx

Getting machine status, if tCRM9250N_DevStatusEx. ByCassettePlugRemove value is 1, shows the cassette been pull out the insert, you need to call “iGetCassetteInfoEx”, after reading the cassette information, byCassettePlugRemove value becomes 0.

int iGetLastDevStatusEx(HANDLE p_hDevHandle, tCRM9250N_DevStatusEx *p_psDevStatus);
p_hDevHandle[in] Logical device handle
p_psDevStatus[out] Machine status

4.12 iPrepareNextTransaction

Prepare for the next Transaction.

int iPrepareNextTransaction(HANDLE p_hDevHandle, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psStatus [out] Return status structure.

4.13 iPrepareDeposit

Reserve deposit.

int iPrepareDeposit(HANDLE p_hDevHandle, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psStatus [out] Return status structure.

4.14 iOpenShutter

Open the shutter.

int iOpenShutter(HANDLE p_hDevHandle, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psStatus [out] Return status structure.

4.15 iCloseShutter

Close the shutter.

int iCloseShutter(HANDLE p_hDevHandle, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psStatus [out] Return status structure.

4.16 iGetLastTranDetail

After the device sudden power failure, when device is powered on again, it is called before "iInitEx" to get the number of notes in/out of each part of the last transaction.

int iGetLastTranDetail(HANDLE p_hDevHandle, tCRM9250N_LastInOutUnitsInfo *p_psTranData,BYTE *p_byOperator,tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psTranData[out] Number of notes in/out of each part.
p_byOperator[out] The operation when the power is off.
0. Counting
1. Deposit
2. Cancel the deposit
3. Dispense
4. Audit RC to ATC
5. Audit ATC to RC
6. Retract NF to LRB
7. Retract NF to NE
8. Retract NE
9. Retract NF
10. Initialization
p_psStatus [out] Return status structure.

4.17 iCashCount

Counting. The unqualified notes are returned to NF-outlet and the qualified notes enter the NE temporary storage area.

int iCashCount(HANDLE p_hDevHandle, tCRM9250N_CashCount *p_psCashCount, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psCashCount [in out] Count information
p_psStatus [out] Return status structure.

4.18 iStoreMoneyEx

To deposit Notes into cassettes.

int iStoreMoneyEx(HANDLE p_hDevHandle, tCRM9250N_StoreMoneyEx *p_psStoreMoney, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle

p_psStoreMoney[in out] Deposit information
p_psStatus [out] Return status structure.

4.19 iCancelDepositEx

Cancel the deposit and the notes in NE are returned to NF-outlet.

int iCancelDepositEx(HANDLE p_hDevHandle, tCRM9250N_CancelDepositEx *p_psCancelDeposit, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psCancelDeposit[in out] Cancel deposit information
p_psStatus [out] Return status structure.

4.20 iDispenseEx

Dispense notes from each cassette to NF as required.

int iDispenseEx(HANDLE p_hDevHandle, tCRM9250N_DispenseInfoEx *p_psDispenseInfo, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psDispenseInfo [in out] Dispense information
p_psStatus [out] Return status structure.

4.21 iRetractEx

Rectract NF/NE notes to designated cassette as required.

int iRetractEx(HANDLE p_hDevHandle, tCRM9250N_RetractInfoEx *p_psRetractInfo, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psRetractInfo [in out] Information on the recovery of notes.
p_psStatus [out] Return status structure.

4.22 iGetDenominationTable

Get NV supported currency/denomination.

int iGetDenominationTable(HANDLE p_hDevHandle, tCRM9250N_DenominationTable *p_psDenominationTable, tDevReturn *p_psStatus);

p_hDevHandle[in] Logical device handle
p_psDenominationTable[out] Note denomination information
p_psStatus [out] Return status structure.

4.23 iGetNvConnected

Test whether the NV is connected.

int iGetNvConnected(HANDLE p_hDevHandle, tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psStatus [out] Return status structure.

4.24 iGetNvNotesInfoEx

Get the information of notes recognized by NV module.

int iGetNvNotesInfoEx(HANDLE p_hDevHandle, tGRGNoteDetails *p_psNoteDetails,
tGRGNoteOCRInfo *p_psNoteOCRInfo,UINT *p_puiNumOfItems, BYTE p_byNoteType,
char p_pcFsnFileName[260], tDevReturn *p_psStatus);
p_hDevHandle[in] Logical device handle
p_psNoteDetails[out] Note information
p_psNoteOCRInfo [out] Additional note information
p_puiNumOfItems
[in] Length of the array p_psNoteDetails
[out] The num of note information
p_byNoteType[in] The type of note to get.
0x01- Real notes
0x02- Not real notes
0x03- All notes
0x04- Blacklisted notes
0x08-36 Counterfeit notes (Type 2 notes)
0x10-36 Suspected counterfeit notes (Class 3 notes)
0x20-36 applies to genuine banknotes (Class 4A banknotes)
0x40-36 does not apply to circulation of genuine notes (Class 4B notes)
0x80- Suspicious notes
It supports combinations such as 0x18 for class 2(0x08) and class 3(0x10).
p_pcFsnFileName[out] Path to the generated FSN file
p_psStatus [out] Return status structure.

4.25 iFirmwareDownload

Downloading firmware.

int iFirmwareDownload(HANDLE p_hDevHandle,const char* p_pcDownloadPath, BYTE p_byMode, tDevReturn* p_psStatus);
p_hDevHandle[in] Device handle.
p_pcDownloadPath [in] Firmware file path (directory or individual file)
p_byMode[in] Download mode 1, CE; 2, NV
p_psStatus [out] Return status structure.

4.26 iGetCurFileProgress

Get the current firmware download progress.

int iGetCurFileProgress(HANDLE p_hDevHandle,tCRM9250N_PDLProgress *p_psPdlProgress);
p_hDevHandle[in] Device handle.
p_psPdlProgress[out] Firmware download status

4.27 iGetDevVersionEx

Get the firmware version of each module.

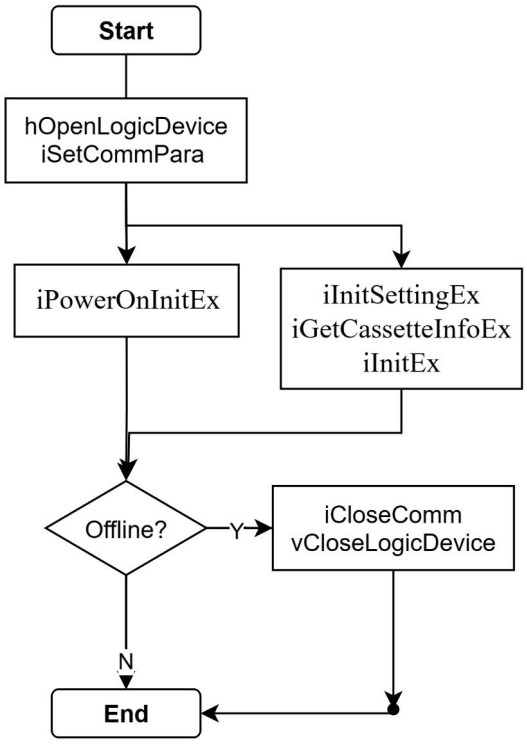
int iGetDevVersionEx(HANDLE p_hDevHandle, tCRM9250N_DevVersionInfoEx *p_psDevVersion, tDevReturn *p_psStatus);
p_hDevHandle[in] Device handle.
p_psDevVersion[out] Firmware version
p_psStatus [out] Return status structure.

4.28 iGetCassetteDenomDetailsEx

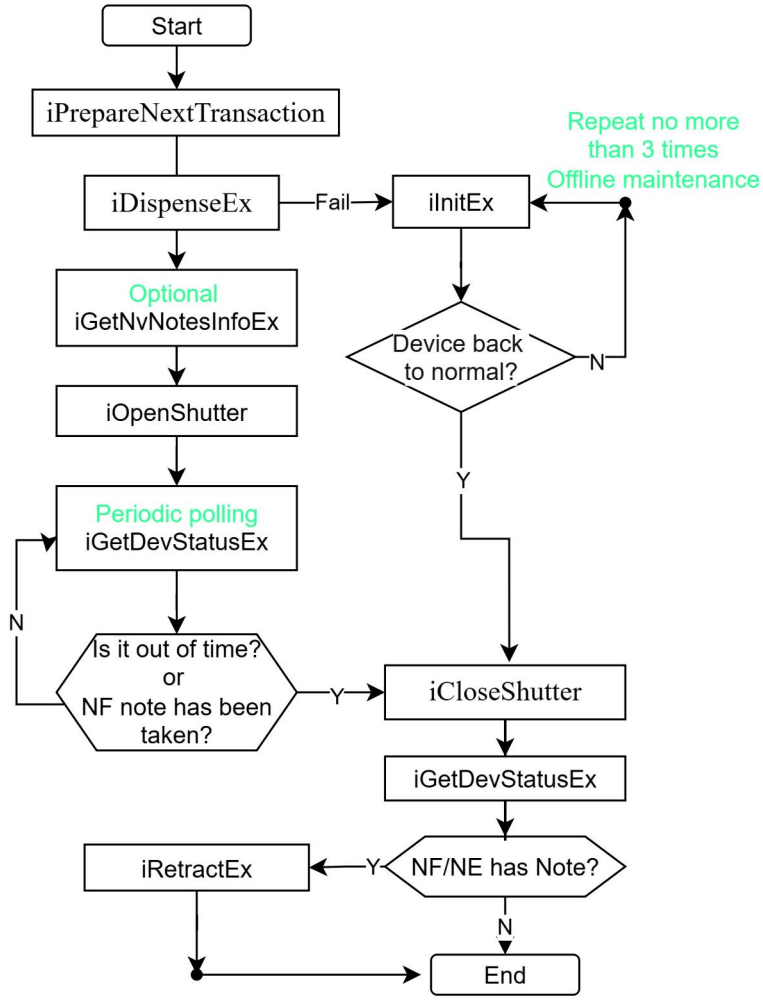
int iGetCassetteDenomDetailsEx(HANDLE p_hDevHandle, int p_iCassetteNo, int *p_piDenomCount, tCRM9250N_DenomDetails *p_psDenomDetails, tDevReturn *p_psStatus);
p_hDevHandle[in] Device handle.
p_iCassetteNo [in] 0: AC1, 1-10: RC1-RC10, 11: LRB, 12: AC2, 13-18: RC11-RC16
p_piDenomCount [in/out]
p_psDenomDetails [Out] Information of currency denomination table
p_psStatus [out] Return status structure.

5 Recommended Flow

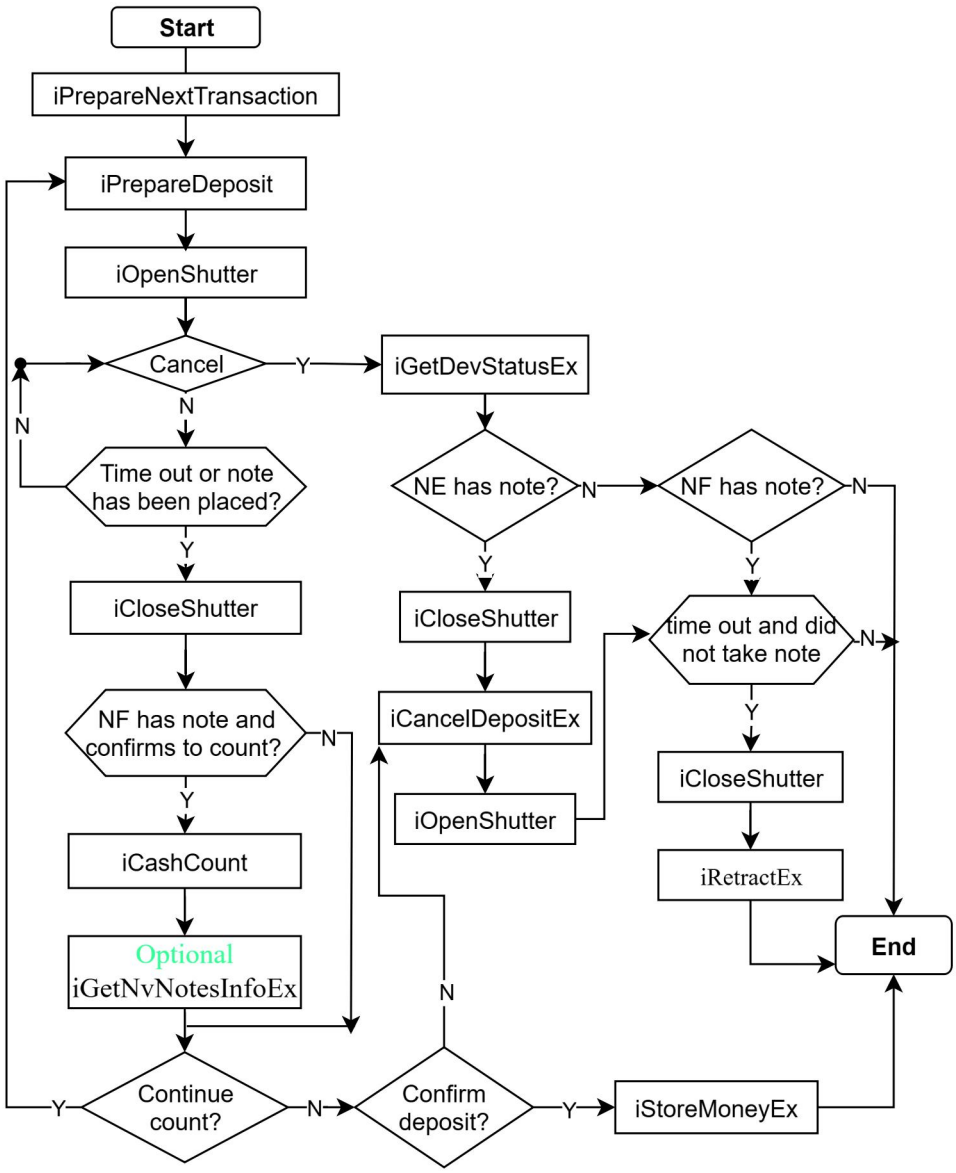
5.1 Power on



5.2 Dispense


Start
iPrepareNextTransaction
iDispenseEx
Fail
iInitEx
Repeat no more than 3 times
Offline maintenance
Optional
iGetNvNotesInfoEx
Device back to normal?
N
iOpenShutter
Y
Periodic polling
iGetDevStatusEx
N
Is it out of time? or NF note has been taken?
Y
iCloseShutter
iGetDevStatusEx
iRetractEx
Y
NF/NE has Note?
N
End

5.3 Deposit


Start
iPrepareNextTransaction
iPrepareDeposit
iOpenShutter
Cancel
Y
N
iGetDevStatusEx
Time out or note has been placed?
N
Y
iCloseShutter
NF has note and confirms to count?
N
Y
iCashCount
Optional
iGetNvNotesInfoEx
Continue count?
Y
N
NE has note?
N
Y
iCloseShutter
iCancelDepositEx
iOpenShutter
NF has note?
N
Y
time out and did not take note
N
Y
iCloseShutter
iRetractEx
Confirm deposit?
N
Y
iStoreMoneyEx
End